Univerza v Ljubljani
Fakulteta za računalništvo in informatiko

Luka Kalin

Optimalno reševanje igre Besedle

diplomsko delo

univerzitetni študijski program
prve stopnje
računalništvo in informatika

Mentor: doc. dr. Tomaž Hočevar

Ljubljana, 2026

To delo je ponujeno pod licenco *Creative Commons Priznanje avtorstva-Deljenje pod enakimi pogoji 2.5 Slovenija* (ali novejšo različico). To pomeni, da se tako besedilo, slike, grafi in druge sestavine dela kot tudi rezultati diplomskega dela lahko prosto distribuirajo, reproducirajo, uporabljajo, priobčujejo javnosti in predelujejo, pod pogojem, da se jasno in vidno navede avtorja in naslov tega dela in da se v primeru spremembe, preoblikovanja ali uporabe tega dela v svojem delu, lahko distribuira predelava le pod licenco, ki je enaka tej. Podrobnosti licence so dostopne na spletni strani creativecommons.si ali na Inštitutu za intelektualno lastnino, Streliška 1, 1000 Ljubljana.



Izvorna koda diplomskega dela, njeni rezultati in v ta namen razvita programska oprema je ponujena pod licenco GNU General Public License, različica 3 (ali novejša). To pomeni, da se lahko prosto distribuira in/ali predeluje pod njenimi pogoji. Podrobnosti licence so dostopne na spletni strani `http://www.gnu.org/licenses/`.

Besedilo je oblikovano z urejevalnikom besedil LaTeX.

Kandidat: Luka Kalin

Naslov: Optimalno reševanje igre Besedle

Vrsta naloge: npr. Diplomska naloga na visokošolskem programu prve stopnje

Mentor: doc. dr. Tomaž Hočevar

Opis:

Besedilo teme diplomskega dela študent prepiše iz študijskega informacijskega sistema, kamor ga je vnesel mentor. V nekaj stavkih bo opisal, kaj pričakuje od kandidatovega diplomskega dela. Kaj so cilji, kakšne metode naj uporabi, morda bo zapisal tudi ključno literaturo.

Title: Optimal Solution of the Besedle Game

Description:

The student shall transcribe the text of the diploma thesis topic from the student information system, where it was entered by the mentor. In a few sentences, they will describe what is expected of the candidate's thesis. What the objectives are, what methods should be used, and perhaps they will also list the key literature.

Zahvaljujem se mentorju prof. dr. Tomažu Hočevarju za strokovno vodenje in pomoč pri izdelavi diplomske naloge.

Kazalo

Povzetek

Abstract

1 Uvod **1**

1.1 Motivacija 1

1.2 Cilji naloge 2

1.3 Struktura naloge 2

2 Sorodna dela **3**

2.1 Optimalna rešitev igre Wordle 3

2.2 Slovenska različica 4

3 Teoretične osnove **5**

3.1 Opis igre Besedle 5

3.2 Model igre 5

3.3 Obarvanje ugibanja 6

3.4 Bellmanova enačba 7

3.5 Optimizacije algoritma 7

4 Implementacija **11**

4.1 Besedišče 11

4.2 Podpora za slovensko abecedo 12

4.3 Izračun obarvanja 12

4.4 Predizračun obarvanj 12

4.5 Memoizacija 13
4.6 Dinamično programiranje 14
4.7 Paralelizacija z OpenMP 15

5 Eksperimenti in rezultati **17**
5.1 Primerjava enostavnejših strategij 17
5.2 Rezultati za igro Besedle 18
5.3 Rezultati za različne dolžine besed 18
5.4 Eksperimentalna potrditev 20
5.5 Analiza časovne zahtevnosti 21

6 Zaključek **23**
6.1 Primerjava z angleško različico 23
6.2 Možne nadgradnje 24

Literatura **25**

Povzetek

Naslov: Optimalno reševanje igre Besedle

Avtor: Luka Kalin

Besedle je popularna slovenska različica besedne igre Wordle, kjer igralec v omejenem številu poskusov ugiba skrito besedo s pomočjo barvnih povratnih informacij. V diplomski nalogi smo implementirali algoritem dinamičnega programiranja, ki za poljuben nabor besed izračuna optimalno strategijo ugibanja z minimalnim pričakovanim številom ugibanj. "Algoritem temelji na Bellmanovi enačbi in vključuje pet optimizacij iz literature. Poleg tega smo implementirali paralelizacijo z OpenMP, zaščito sočasnega dostopa do memoizacijske tabele ter kompaktno bitset predstavitev stanj."Besedišče smo pridobili iz morfološkega leksikona Sloleks 3.0 in neposredno iz igre Besedle. Za igro Besedle z 1861 besedami dolžine 5 smo izračunali optimalno pričakovano število ugibanj $V^* = 3{,}395$ ter določili optimalno začetno besedo *torek*. Algoritem smo posplošili na besede dolžin od 3 do 15 črk in eksperimentalno potrdili pravilnost izračunanih vrednosti.

Ključne besede: Besedle, Wordle, dinamično programiranje, optimalna strategija.

Abstract

Title: Optimal Solution of the Besedle Game

Author: Luka Kalin

Besedle is a popular Slovenian version of the word game Wordle, where a player guesses a hidden word within a limited number of attempts using color-coded feedback. In this thesis, we implemented a dynamic programming algorithm that computes the optimal guessing strategy with minimal expected number of guesses for an arbitrary set of words. The algorithm is based on the Bellman equation and incorporates five optimizations from the literature. Additionally, we implemented parallelization using OpenMP, thread-safe access to the memoization table, and a compact bitset state representation. The word list was obtained from the Sloleks 3.0 morphological lexicon and directly from the Besedle game. For the Besedle game with 1861 five-letter words, we computed the optimal expected number of guesses $V^* = 3.395$ and identified the optimal starting word *torek*. The algorithm was generalized to words of lengths from 3 to 15 letters, and the correctness of the computed values was experimentally confirmed.

Keywords: Besedle, Wordle, dynamic programming, optimal strategy.

Poglavje 1

Uvod

Besedle je popularna slovenska različica besedne igre Wordle, ki jo je razvil Josh Wardle leta 2021. V igri mora igralec v največ šestih poskusih uganiti skrito pet-črkovno besedo. Po vsakem ugibanju dobi povratno informacijo:

- zeleno obarvana črka pomeni, da je črka na pravilnem mestu,
- rumeno obarvana črka pomeni, da je črka v besedi, vendar na napačnem mestu,
- sivo obarvana črka pomeni, da črke ni v iskani besedi.

Igra je postala izjemno priljubljena po vsem svetu, kar je spodbudilo razvoj številnih različic v različnih jezikih. Slovenska različica Besedle je dostopna na spletni strani `https://besedle.com` in vsebuje 1861 možnih rešitev ter 3956 veljavnih ugibanj.

1.1 Motivacija

Optimalno reševanje besednih iger je zanimiv kombinatoričen problem z jasno definirano strukturo. Bertsimas in Paskov [1] sta pokazala, da je mogoče z eksaktnim dinamičnim programiranjem najti certifikatno optimalno rešitev za angleško različico igre Wordle. Motivacija za to diplomsko nalogo je bila

prilagoditev tega pristopa na slovensko različico igre ter posplošitev na besede poljubnih dolžin.

1.2 Cilji naloge

Cilji diplomske naloge so:

- implementacija eksaktnega algoritma dinamičnega programiranja za iskanje optimalne strategije ugibanja,
- izračun optimalne strategije za igro Besedle in določitev optimalne začetne besede,
- posplošitev algoritma na besede poljubnih dolžin in poljubne nabore besed,
- eksperimentalna potrditev pravilnosti izračunanih vrednosti.

1.3 Struktura naloge

Diplomska naloga je organizirana kot sledi. V poglavju 2 predstavimo sorodna dela in obstoječe pristope k reševanju besednih iger. Poglavje 3 opisuje teoretične osnove – model igre in algoritem dinamičnega programiranja. V poglavju 4 opišemo implementacijo algoritma v programskem jeziku C++. Poglavje 5 predstavi rezultate in eksperimentalno potrditev. V poglavju 6 povzamemo ugotovitve in predlagamo možne nadgradnje.

Poglavje 2

Sorodna dela

Wordle in njene različice so leta 2021 spodbudile zanimanje za optimalno reševanje besednih iger. Bertsimas in Paskov [1] sta pokazala, da je mogoče z eksaktnim dinamičnim programiranjem najti certifikatno optimalno rešitev.

2.1 Optimalna rešitev igre Wordle

Bertsimas in Paskov [1] sta razvila eksaktni algoritem za optimalno reševanje angleške različice igre Wordle. Njun pristop temelji na dinamičnem programiranju z Bellmanovo enačbo, ki za vsako možno stanje igre izračuna optimalno pričakovano število ugibanj. Algoritem vključuje pet optimizacij [1, str. 7–9], ki drastično zmanjšajo računsko zahtevnost: skrajšanje horizonta, robni pogoj za dve besedi, predizračun obarvanj, orezovanje s spodnjimi mejami ter prednostno preizkušanje kandidatov.

Za angleško različico Wordle z 2315 možnimi rešitvami in 10657 veljavnimi ugibanj sta avtorja izračunala optimalno pričakovano število ugibanj $V^* = 3{,}421$ z optimalno začetno besedo *SALET*. Izračun je zahteval večdnevno računanje na 64-jedrnem strežniku.

2.2 Slovenska različica

Eksaktna optimalna strategija za slovensko različico igre Besedle pred to diplomsko nalogo ni bila objavljena. Naša rešitev se od angleškega pristopa [1] razlikuje v:

- prilagoditvi za slovensko abecedo z znaki š, č, ž,
- uporabi slovenskega besedišča iz morfološkega leksikona Sloleks 3.0 ter neposredno iz igre Besedle [3],
- posplošitvi na besede poljubnih dolžin (3–15 črk),
- dodatnih optimizacijah (paralelizacija z OpenMP, zaščita sočasnega dostopa do memoizacijske tabele in bitset predstavitev stanj).

Poglavje 3

Teoretične osnove

3.1 Opis igre Besedle

Besedle je besedna igra v kateri mora igralec v največ šestih poskusih uganiti skrito besedo. Po vsakem ugibanju dobi barvno povratno informacijo kot je opisano v poglavju 1. V osnovni različici igre so vse besede dolžine pet črk. Igra vsebuje dva seznama besed:

- seznam rešitev – besede ki so lahko skrita beseda (1861 besed),
- seznam ugibanj – vse veljavne besede ki jih smemo ugibati (3956 besed).

3.2 Model igre

Igro modeliramo kot *Markovov odločitveni proces* (MDP) – matematični okvir za modeliranje odločanja v situacijah kjer rezultat akcije ni popolnoma predvidljiv. Ključno je da stanje igre ni celotna zgodovina ugibanj, ampak samo množica besed ki so še možne rešitve. To bistveno zmanjša prostor stanj – namesto eksponentnega števila možnih zgodovin ugibanj imamo samo podmnožice slovarja.

Formalno definiramo:

- Stanje s – množica besed ki so še možne rešitve glede na dosedanje informacije. Začetno stanje je celoten slovar rešitev (1861 besed za Besedle), končno stanje je množica z eno besedo ko ugodimo. Ta definicija zadošča Markovski predpostavki – znanje o prihodnjih stanjih je odvisno samo od trenutne množice veljavnih rešitev.

- Akcija a – beseda ki jo ugibamo, izbrana iz celotnega slovarja veljavnih ugibanj $\mathcal{A}$. Za Besedle vsebuje $|\mathcal{A}| = 3956$ besed.

- Funkcija stroškov – vsako ugibanje stane 1, izguba igre stane ∞. Formalno: $c_t(s, a) = 1$ za $t \leq 5$ in $c_6(s) = \infty$. Če ima strategija končen strošek, zagotovo zmaga ne glede na skrito besedo.

- Prehod – za dano stanje s in ugibanje a je možnih največ 3^n naslednih stanj (kjer je n dolžina besede), ker obstaja toliko možnih obarvanj. Novo stanje vsebuje vse besede ki bi dale enako obarvanje:

$$s' = \{w \in s \mid \text{obarvanje}(a, w) = c\} \tag{3.1}$$

  Verjetnost prehoda v stanje s' je $\frac{|s'|}{|s|}$.

- Cilj – poiskati strategijo ugibanja ki minimizira pričakovano število ugibanj do konca igre.

3.3 Obarvanje ugibanja

Funkcija obarvanja obarvanje(a, w) vrne zaporedje barv za ugibanje a glede na skrito besedo w. Izračun poteka v dveh korakih:

1. Zelene črke – najprej označimo vse črke ki so na pravilnem mestu.

2. Rumene črke – nato označimo preostale črke ki obstajajo v besedi ampak na napačnem mestu.

Dvostopenjski postopek je potreben za pravilno obravnavo podvojenih črk.

3.4 Bellmanova enačba

Optimalno pričakovano število ugibanj $V^*(s)$ za stanje s zadošča Bellmanovi enačbi:

$$V^*(s) = \min_{a \in \mathcal{A}} \left\{ 1 + \sum_{s' \in \mathcal{S}(s,a)} \frac{|s'|}{|s|} \cdot V^*(s') \right\} \tag{3.2}$$

kjer je $\mathcal{A}$ množica vseh veljavnih ugibanj in $\mathcal{S}(s,a)$ množica možnih naslednih stanj po ugibanju a.

Robni pogoji:

$$V^*(\{w\}) = 1 \tag{3.3}$$

$$V^*(\{w_1, w_2\}) = 1{,}5 \tag{3.4}$$

Enačba (3.3) velja ker pri enem kandidatu takoj uganeš. Enačba (3.4) velja ker pri dveh kandidatih ne glede na izbiro ugibanja v povprečju potrebuješ 1,5 ugibanja – z verjetnostjo $\frac{1}{2}$ uganeš takoj, z verjetnostjo $\frac{1}{2}$ pa potrebuješ še eno ugibanje.

3.5 Optimizacije algoritma

Direktna implementacija Bellmanove enačbe bi bila računsko neizvedljiva. Bertsimas in Paskov [1] v svojem algoritmu uporabita pet optimizacij ki drastično zmanjšajo računsko zahtevnost:

3.5.1 Optimizacija 1: Skrajšanje horizonta

Igra Besedle dovoljuje največ šest ugibanj. Če na zadnji potezi ostane več kot en kandidat, strategija ne more zmagati z gotovostjo:

$$V^*(s,t) = \infty \quad \text{če } t \geq 6 \text{ ali } (t = 5 \text{ in } |s| > 1) \tag{3.5}$$

Ta opazka efektivno skrajša časovni horizont igre za 1 – algoritem namesto šestih potez dejansko išče rešitve v petih, saj mora biti na peti potezi zagotovo znan zmagovalec. To bistveno zmanjša prostor iskanja.

3.5.2 Optimizacija 2: Robni pogoj za dva kandidata

Ko ostaneta natanko dva kandidata $\{w_1, w_2\}$, je vrednost vedno $V^* = 1{,}5$ ne glede na to kateri dve besedi sta. To prepreči nepotrebno rekurzijo.

Ne glede na to katero besedo ugibamo:

- z verjetnostjo $\frac{1}{2}$ ugodimo takoj $\rightarrow$ 1 ugibanje,
- z verjetnostjo $\frac{1}{2}$ ostane ena beseda $\rightarrow$ še 1 ugibanje.

$$V^*(\{w_1, w_2\}) = 1 \times \frac{1}{2} + 2 \times \frac{1}{2} = 1{,}5 \tag{3.6}$$

3.5.3 Optimizacija 3: Predizračun obarvanj

Funkcija obarvanja se kliče večmilijonkrat med izvajanjem algoritma. Vnaprej izračunamo tabelo vseh kombinacij ugibanje $\times$ kandidat:

$$T[i][j] = \text{obarvanje}(\text{ugibanje}_i, \text{kandidat}_j) \tag{3.7}$$

Med algoritmom namesto računanja samo pogledamo vrednost v tabeli.

Za igro Besedle je tabela velikosti $3956 \times 1861 \approx 7{,}4$ milijona vnosov. Predizračun traja nekaj sekund, nato pa vsak dostop do obarvanja poteka v konstantnem času namesto linearnega računanja čez vse znake besede.

3.5.4 Optimizacija 4: Obrezovanje s spodnjimi mejami

Spodnja meja vrednosti stanja z n kandidati je:

$$\underline{V}(n) = \frac{2n - 1}{n} \tag{3.8}$$

Intuitivno to pomeni najboljši možni scenarij – z verjetnostjo $\frac{1}{n}$ ugodimo takoj, z verjetnostjo $\frac{n-1}{n}$ pa potrebujemo točno še eno ugibanje. Dejanska vrednost bo vedno enaka ali višja.

Obrezovanje poteka v dveh korakih:

1. Za vsako naslednje stanje s' najprej uporabimo spodnjo mejo $\underline{V}(|s'|)$ namesto točne vrednosti. Če je že ta ocena slabša od trenutnega minimuma, ugibanje preskočimo brez rekurzije.

2. Med računanjem sproti zamenjujemo spodnje meje z eksaktnimi vrednostmi. Takoj ko vsota preseže trenutni minimum, prekinemo računanje tega ugibanja.

Naslednja stanja z večjo verjetnostjo prehoda obravnavamo najprej, saj najbolj vplivajo na skupno vsoto in omogočijo zgodnejše orezovanje.

3.5.5 Optimizacija 5: Prioriteta kandidatom

Med ugibanjem najprej preizkusimo besede iz množice kandidatov s, šele nato ostale besede iz slovarja $\mathcal{A}$. Bertsimas in Paskov [1] sta empirično ugotovila, da je optimalno ugibanje a^* v večini primerov beseda iz trenutne množice kandidatov ($a^* \in s_t$).

Ta ureditev vrstnega reda izkorisča optimizacijo 4 – če dober kandidat najdemo zgodaj, pruning hitreje odreže slabša ugibanja iz preostalega slovarja.

Poglavje 4

Implementacija

Algoritem smo implementirali v programskem jeziku C++, ki zagotavlja visoko zmogljivost pri rekurzivnih izračunih in podpira paralelizacijo z OpenMP.

4.1 Besedišče

4.1.1 Sloleks 3.0

Za slovensko besedišče smo uporabili morfološki leksikon Sloleks 3.0 [4], ki vsebuje 365.340 vnosov slovenskega knjižnega jezika. Iz leksikona smo ekstrahirali:

- samo knjižne besede (ne narečnih),
- samo občna imena (ne lastnih imen),
- samo samostalnike in pridevnike,
- samo osnovne oblike (leme).

Besede smo razdelili v dva seznama glede na frekvenco v korpusu Gigafida 2.0 [2]:

- seznam rešitev – besede s frekvenco ≥ 200 (pogostejše besede),
- seznam ugibanj – besede s frekvenco ≥ 1 (vse knjižne besede).

4.1.2 Besedle

Poleg Sloleks baze smo pridobili tudi dejanski nabor besed iz igre Besedle [3], ki vsebuje 1861 možnih rešitev in 3956 veljavnih ugibanj, vse dolžine 5 črk. Besede so bile pridobljene neposredno iz izvorne kode spletne strani.

4.2 Podpora za slovensko abecedo

Slovenščina vsebuje šumnike š, č, ž, ki v kodiranju UTF-8 zavzamejo 2 bajta namesto 1. Standardni C++ niz (`std::string`) obravnava bajte, ne znake, kar bi povzročilo napake pri primerjavi črk.

Implementirali smo funkcijo `utf8_v_unicode`, ki pretvori UTF-8 niz v zaporedje 32-bitnih Unicode kodnih točk. S tem zagotovimo, da vsak element ustreza točno enemu znaku:

- enočrkovna znaka (ASCII): 1 bajt $\rightarrow$ 1 kodna točka,
- šumniki (š, č, ž): 2 bajta $\rightarrow$ 1 kodna točka.

4.3 Izračun obarvanja

Funkcija `get_coloring` izračuna obarvanje ugibanja glede na skrito besedo. Implementirana sta oba koraka opisana v poglavju 3: najprej zelene, nato rumene črke. Za pravilno obravnavo podvojenih črk sledimo kateri znaki so že bili »porabljeni«.

4.4 Predizračun obarvanj

Pred začetkom algoritma izračunamo tabelo vseh kombinacij obarvanj. Za n_u ugibanj in n_k kandidatov je tabela velikosti $n_u \times n_k$. Za igro Besedle:

$$3956 \times 1861 \approx 7{,}4 \text{ milijona vnosov} \tag{4.1}$$

Predizračun traja nekaj sekund, nato pa vsak dostop do obarvanja poteka v konstantnem času namesto linearnega računanja čez vse znake besede.

4.5 Memoizacija

Za shranjevanje že izračunanih vrednosti $V^*(s)$ smo implementirali memoizacijo z zgoščevalno tabelo (`std::unordered_map`). Zgoščevalna tabela omogoča iskanje in vstavljanje v povprečnem času $O(1)$, kar je ključno za učinkovitost algoritma.

Tabela preslika ključ stanja (množico kandidatov) v izračunano vrednost V^*:

$$\text{memo}[\text{ključ}(s)] = V^*(s) \tag{4.2}$$

Pred vsakim rekurzivnim izračunom preverimo ali je stanje že v tabeli – če je, takoj vrnemo shranjeno vrednost brez nadaljnje rekurzije. Za igro Besedle (1861 besed) je memo tabela ob koncu izračuna vsebovala 157 milijonov vnosov, od tega je bilo 157 milijonov zadetkov – torej je bila rekurzija prihranjena tolikokrat.

4.5.1 Bitset ključ

Ključna lastna optimizacija je kompaktna bitset predstavitev stanja. Vsaka beseda v slovarju dobi enolični indeks. Stanje (množica kandidatov) predstavimo kot bitni niz dolžine n bitov, kjer n je velikost slovarja:

- bit i je nastavljen na 1, če je beseda z indeksom i še možna rešitev,
- bit i je nastavljen na 0, sicer.

Za 1861 besed potrebujemo $\lceil 1861/8 \rceil = 233$ bajtov na ključ. Prednosti bitset ključa:

- Neodvisnost od vrstnega reda – ista množica kandidatov vedno da enak ključ,
- Kompaktnost – fiksna velikost 233 bajtov ne glede na število kandidatov,
- Hitrost primerjave – primerjava ključev je hitra.

4.5.2 Thread-safe memoizacija

Ker algoritem teče vzporedno na več nitih (OpenMP), morata biti branje in pisanje v memo tabelo zaščitena z vzajemnim izključevanjem (`std::mutex`). Brez zaščite bi lahko prišlo do t.i. *race condition* – situacije kjer dve niti hkrati pišeta v isto lokacijo in pokvarita podatke.

Primer brez zaščite:

- nit 1 preveri: ključ ni v memo tabeli
- nit 2 preveri: ključ ni v memo tabeli
- nit 1 izračuna $V^* = 2.5$ in zapiše v tabelo
- nit 2 izračuna $V^* = 2.5$ in zapiše čez ← podvojen izračun ali pokvarjeni podatki!

Zaščito implementiramo z `std::lock_guard` ki zaklep avtomatsko sprosti ob izhodu iz bloka – tudi v primeru napake:

- pred branjem iz memo: zakleni → preberi → sprosti
- pred pisanjem v memo: zakleni → zapiši → sprosti

Med samim računanjem V^* zaklep ni aktiven – niti računajo neodvisno in se blokirajo samo ob dostopu do skupne memo tabele. To minimizira čakanje med nitmi.

4.6 Dinamično programiranje

Osrednja funkcija `compute_V` rekurzivno izračuna $V^*(s)$ z vsemi optimizacijami:

1. Preverimo robne pogoje (optimizaciji 1 in 2).
2. Preverimo memo tabelo – če je stanje že izračunano, takoj vrnemo vrednost.

3. Za vsako možno ugibanje izračunamo spodnjo mejo (optimizacija 4).
4. Če spodnja meja presega trenutni minimum, ugibanje preskočimo (pruning).
5. Sicer rekurzivno izračunamo točno vrednost.
6. Shranimo rezultat v memo tabelo.

4.7 Paralelizacija z OpenMP

Zunanja zanka čez vsa možna ugibanja je paralelizirana z direktivo `#pragma omp parallel for schedule(dynamic, 1)`. Vsaka nit vzdržuje lasten lokalni minimum, ki ga primerjamo z globalnim minimumom znotraj `#pragma omp critical` sekcije – s tem se izognemo dirkalnim pogojem pri posodabljanju skupne vrednosti.

Dinamično razporejanje (`schedule(dynamic, 1)`) zagotovi enakomerno obremenitev jeder: ker se čas izračuna posameznega ugibanja močno razlikuje (odvisno od tega, kako hitro algoritem obreže veje), bi statična razdelitev dela povzročila, da nekatere niti čakajo na počasnejše. Z dinamičnim razporejanjem se ugibanja nitim dodelujejo sproti glede na razpoložljivost.

Implementacija je bila preizkušena na strežniku s 6 jedri in na lokalnem računalniku z 8 jedri.

Poglavje 5

Eksperimenti in rezultati

Eksperimente smo izvajali na dveh sistemih:

- Lokalni računalnik – AMD Ryzen 7 9700X (8 jeder), 32 GB RAM, Windows 11.
- Fakultetni strežnik – 6 jeder, 59 GB RAM, Ubuntu 20.04.

Algoritem smo prevedli z ukazom:

```
g++ -O3 -fopenmp -o wordle_solver wordle_solver.cpp
```

5.1 Primerjava enostavnejših strategij

Pred implementacijo optimalnega algoritma smo evalvirali dve enostavnejši strategiji za 5-črkovne besede Besedle (500 besed):

- Naključna strategija – iz preostalih kandidatov naključno izbere ugibanje.
- Hevristična strategija – izbere besedo z največ pogostimi črkami med preostalimi kandidati.

Tabela 5.1: Primerjava strategij za 5-črkovne besede Besedle (500 besed)

Strategija	Povprečno število ugibanj
Naključna	4,02
Hevristična	3,70
DP optimalna	**3,002**

5.2 Rezultati za igro Besedle

Za igro Besedle z 1861 besedami dolžine 5 smo izračunali:

- Optimalno pričakovano število ugibanj: $V^* = 3{,}395$
- Optimalna začetna beseda: TOREK
- Čas izračuna: 129 ur (fakultetni strežnik, 6 jeder)

Za primerjavo – angleška različica Wordle ima $V^* = 3{,}421$ z optimalno začetno besedo *SALET* pri 2315 možnih rešitvah [1]. Slovenska igra Besedle je torej rahlo lažja od angleške.

Omeniti velja, da smo tako pri ugibanjih kot pri rešitvah uporabili isti nabor 1861 besed. Polni slovar Besedle vsebuje tudi 3956 besed, ki so veljavna ugibanja, a ne morejo biti rešitev. Vključitev celotnega slovarja bi verjetno rahlo znižala V^*, vendar bi čas izračuna presegel zmogljivosti uporabljene strojne opreme.

5.3 Rezultati za različne dolžine besed

Algoritem smo posplošili na besede dolžin od 3 do 15 črk iz leksikona Sloleks 3.0. Število dovoljenih ugibanj smo prilagodili glede na dolžino besed.

Tabela 5.2: Rezultati za besede različnih dolžin

Dolžina	Besed	Max potez	V^*	Optimalni začetek	Čas
3	200	9	3,985	sok	39 min
3	300	9	4,347	dok	3,2 ure
4	300	8	3,317	kalo	16 min
4	500	8	3,558	kalo	5,1 ure
5	500	6	3,002	tarok	21,5 s
5	1861	6	3,395	torek	129 ur
6	2566	7	3,080	krtina	7,4 ure
7	2943	7	2,863	karoten	1,5 ure
8	2926	6	2,692	klarinet	2,1 min
9	2684	6	2,561	podaritev	34,5 s
10	2164	5	2,414	polresnica	8,5 s
11	1613	5	2,280	redovalnica	2,3 s
12	1128	4	2,176	narkolepsija	0,76 s
13	691	4	2,100	razpredelnica	0,19 s
14	385	3	2,034	nesorazmernost	0,03 s
15	203	3	2,005	profesionalizem	0,006 s

Opazimo tri jasne tendence:

- Daljše besede $\rightarrow$ nižji V^* – z večjo dolžino besed vsako ugibanje da več informacije (3^n možnih obarvanj za n-črkovno besedo), kar pospeši konvergenco.

- Večji nabor $\rightarrow$ višji V^* – večji slovar pomeni težjo igro.

- Daljše besede $\rightarrow$ hitrejše računanje – manjše število potrebnih potez drastično zmanjša prostor stanj.

5.4 Eksperimentalna potrditev

Za potrditev pravilnosti algoritma smo simulirali igre z optimalno strategijo in primerjali povprečno število ugibanj z izračunanim V^*.

Optimalna strategija za simulacijo na vsakem koraku izbere ugibanje, ki minimizira vrednost V^* naslednjega stanja. Vrednosti so že shranjene v memo tabeli iz predhodnega izračuna, zato simulacija ne zahteva ponovnega računanja.

Simulacija se od izračunanega optimuma razlikuje v enem ključnem vidiku: pri iskanju najboljšega ugibanja upošteva samo preostale kandidate, medtem ko DP optimizira čez celoten nabor besed. Izračunani optimum tako v nekaterih primerih predpisuje ugibanje besede, ki zagotovo ni med preostalimi rešitvami, zgolj z namenom pridobivanja informacij o preostalih kandidatih. Simulacija te možnosti ne upošteva in ugiba le med preostalimi kandidati, kar je bolj naravno za človeškega igralca.

Tabela 5.3: Primerjava izračunanega in eksperimentalnega V^*

Dolžina	Besed	V^* izračunan	V^* eksperiment	Razlika
3	200	3,985	4,150	0,165
4	300	3,317	3,357	0,040
5	500	2,988	2,990	0,002
6	2566	3,080	3,137	0,057
7	2943	2,863	2,878	0,014
8	2926	2,692	2,707	0,015
9	2684	2,561	2,568	0,007
10	2164	2,414	2,418	0,004
11	1613	2,280	2,280	≈ 0
12	1128	2,176	2,176	≈ 0
13	691	2,100	2,100	≈ 0
14	385	2,034	2,034	≈ 0
15	203	2,005	2,005	≈ 0

Za besede dolžin 11–15 je razlika praktično nič (reda 10^{-14}), kar je zgolj zaokrožitvena napaka. Za krajše besede je razlika nekoliko večja, ker simulacija ugiba le med preostalimi kandidati in ne more izkoristiti informacijskih ugibanj iz celotnega nabora besed.

5.5 Analiza časovne zahtevnosti

Časovna zahtevnost algoritma eksponentno raste z velikostjo nabora besed. Iz eksperimentalnih rezultatov opazimo:

- Za 5-črkovne besede: 200 $\rightarrow$ 0,13 s, 500 $\rightarrow$ 14 s, 1000 $\rightarrow$ 27 min, 1861 $\rightarrow$ 129 ur.
- Podvojitev nabora besed poveča čas izračuna za faktor 10–100$\times$.

Razlog za tako hitro rast tiči v strukturi problema: vsako ugibanje razdeli preostale kandidate v do 3^n skupin (kjer je n dolžina besede), algoritem pa mora rekurzivno rešiti vsako skupino. Skupno število stanj v memo tabeli tako eksponentno narašča z velikostjo nabora.

Za polni nabor besed Besedle (1861 besed) je bil izračun izveden na fakultetnem strežniku s 6 jedri v 129 urah. Avtorji originalnega članka [1] so za angleško različico z 2315 besedami potrebovali več dni na 64-jedrnem strežniku, kar potrjuje skladnost naših rezultatov.

V praksi je algoritem izvedljiv za nabore do nekaj tisoč besed, za večje nabore pa bi bila potrebna zmogljivejša strojna oprema.

Poglavje 6

Zaključek

V diplomski nalogi smo implementirali optimalni algoritem za reševanje igre Besedle z uporabo dinamičnega programiranja. Pokazali smo, da je mogoče z Bellmanovo enačbo natančno izračunati optimalno pričakovano število ugibanj V^* za poljuben nabor besed.

Za uradni nabor igre Besedle s 1861 petčrkovnimi besedami smo določili optimalno vrednost $V^* = 3{,}395$ in ugotovili, da je optimalna začetna beseda *torek*. Izračun je bil izveden v 129 urah na strežniku s 6 jedri, kar je skladno z rezultati za angleško različico igre Wordle.

Eksperimentalna primerjava z naključno in hevristično strategijo je potrdila, da dinamično programiranje zagotavlja bistveno boljše rezultate. Analiza za besede različnih dolžin je pokazala, da se optimalno pričakovano število ugibanj zmanjšuje z dolžino besed, kar je posledica manjšega razmerja med velikostjo nabora in številom možnih barvanj.

Glavna omejitev pristopa je računska zahtevnost, ki eksponentno narašča z velikostjo nabora besed. Za večje nabore bi bila potrebna zmogljivejša strojna oprema ali drugačni pristopi.

6.1 Primerjava z angleško različico

Primerjava z rezultati za angleško različico Wordle [1]:

- Angleška različica: $V^* = 3{,}421$, optimalna začetna beseda *SALET*.
- Slovenska različica: $V^* = 3{,}395$, optimalna začetna beseda *torek*.

Slovenska igra Besedle je rahlo lažja od angleške, kar je verjetno posledica manjšega nabora besed (1861 vs. 2315).

6.2 Možne nadgradnje

Diplomska naloga odpira več možnosti za nadaljnje delo:

- Polni slovar ugibanj – v tej nalogi smo za slovar ugibanj uporabili samo seznam rešitev. Uporaba celotnega slovarja ugibanj (3956 besed) bi algoritem sprostila pri izbiri informacijskih ugibanj in bi verjetno znižala V^*, a bi zahtevala bistveno več računskega časa.
- Večja paralelizacija – z dostopom do strežnika z več jedri bi lahko izračun za večje nabore besed izvedli v sprejemljivem času.
- Interpretabilna strategija – Bertsimas in Paskov [1] sta optimalno strategijo predstavila z odločitvenimi drevesi. Podoben pristop bi lahko aplicirali na slovensko različico.

Literatura

[1] Dimitris Bertsimas in Alex Paskov. „An Exact and Interpretable Solution to Wordle“. V: (2022). URL: https://auction-upload-files.s3.amazonaws.com/Wordle_Paper_Final.pdf (pridobljeno 29. 9. 2026).

[2] *Gigafida 2.0 – korpus slovenskega jezika*. 2019. URL: https://www.clarin.si/repository/xmlui/handle/11356/1230 (pridobljeno 7. 10. 2026).

[3] MuciBuci1. *Besedle – slovenska različica igre Wordle*. 2022. URL: https://besedle.com (pridobljeno 29. 9. 2026).

[4] *Sloleks 3.0 – morfološki leksikon slovenščine*. 2021. URL: https://www.clarin.si/repository/xmlui/handle/11356/1745 (pridobljeno 29. 9. 2026).